

build-aar.sh

Builds the Android .aar library archive for the lava-api-go embedded server surface (internal/mobile) using gomobile bind. Phase B2 of the Lava API Android app plan.

Script: lava-api-go/scripts/build-aar.sh Output: lava-api-go/build/lavaapi.aar (gitignored — never committed)

What it does

1. Discovers the Android SDK (ANDROID_HOME) and NDK (ANDROID_NDK_HOME), preferring NDK 25.1.8937393, then 21.4.7075529, then any installed NDK.
2. Installs gomobile + gobind user-locally (no sudo) if absent, then runs gomobile init.
3. Runs:

```
gomobile bind -target=android -androidapi 28 \  
-javapkg digital.vasic.lava.apigo \  
-o build/lavaapi.aar ./internal/mobile
```

prefixed with GOMAXPROCS=2 nice -n19 (\$6.T.2 resource limits).
4. Prints the resulting .aar size in bytes and sha256.

Usage

```
cd lava-api-go  
./scripts/build-aar.sh
```

Environment overrides (all optional)

Variable	Default	Purpose
ANDROID_HOME	~/Library/Android/sdk	Android SDK root

Variable	Default	Purpose
ANDROID_NDK_HOME	discovered under \$ANDROID_HOME/ndk	NDK root for the cgo cross- compile
LAVAAPI_JAVAPKG	digital.vasic.lava.apigo	Java package of generated bindings
LAVAAPI_ANDROIDAPI	28	minSdk for the bind

Exit codes

Code	Meaning
0	.aar built; size + sha256 printed
1	bind ran but the .aar is missing (unexpected)
2	configuration error (SDK/NDK not found)

Generated API

gobind exposes the internal/mobile package as a Mobile Java class under digital.vasic.lava.apigo:

- `Mobile.start(String configJSON)` — throws on failure (Go error).
configJSON is
`{"bindAddr":"127.0.0.1","port":8099,"sqlitePath":"/data/x.db"}`.
- `Mobile.stop()` — throws when nothing is running.
- `Mobile.status()` — returns a JSON String
`{"state","bindAddr","port","requestCount","backend","version"}`.

Only string/error types cross the boundary, by design — gobind binds these cleanly to `java.lang.String` and Java exceptions.

Constitutional notes

- **\$6.T.2** — the bind compile is resource-limited (`GOMAXPROCS=2` nice -n19).
- **\$6.U** — no sudo/su; gomobile/gobind install to the user Go bin.
- **\$6.R** — NDK/SDK paths come from env vars with discovery fallbacks; no host-specific path is hardcoded in the script.

- §11.4.18 — this file is the mandated external doc for the script.

Known environment requirement

`gomobile bind` requires a working Android NDK with a C cross-compiler for the Android ABIs. If the bind fails, the verbatim error names the missing piece (NDK toolchain, aar packaging tool, etc.). The `internal/mobile` Go package is independently unit-tested (`go test ./internal/mobile/`) so the surface is verified even when the bind toolchain is unavailable on a given host.

KNOWN BLOCKER (2026-06-02): gomobile bind vs. this module's dependency graph

On the development host, `gomobile bind` FAILS for this module. It is BLOCKED, not faked. The script + doc + the `internal/mobile` package still ship because the package is fully unit-tested independently of the bind toolchain.

Verbatim failure (both NDK 25.1.8937393 AND NDK 21.4.7075529 tried; identical):

```
==> gomobile bind
gomobile: go mod tidy failed: exit status 1
go: error reading go.mod: missing module declaration. To specify
the module path:
    go mod edit -module=example.com/mod
```

Diagnosis (with `gomobile bind -work`): `gomobile` writes a **0-byte** overlay `go.mod` into its temp work dir (`.../gomobile-work-XXXX/src-android-arm64/go.mod`) and then its internal `go mod tidy` chokes on the empty file. Confirmed:

- `gomobile bind` succeeds on a TRIVIAL standalone module (produced a 1.5 MB `.aar`) with the SAME `gomobile` + NDK + Go 1.26.2 toolchain — so the toolchain/NDK are healthy.
- It fails on `lava-api-go` because `internal/mobile` transitively imports `digital.vasic.cache` and `digital.vasic.observability`, which are behind 16 relative `replace` directives to `../submodules/*`. `gomobile`'s overlay-module generation cannot reconstruct this `replace-heavy` graph (the temp work dir has no `../submodules`), and emits an empty overlay `go.mod`.
- Rewriting the relative `replaces` to ABSOLUTE paths did NOT help — `gomobile` still emits the 0-byte overlay `go.mod`, so the limitation is

in gomobile's module-graph templating for replace-using modules, not the path form.

gomobile version: `golang.org/x/mobile v0.0.0-20260529142300-ecb4cd65260a`.

Remediation options (none applied yet — owed to a follow-up phase)

1. **Vendor / flatten the embed deps.** Give `internal/mobile` its own minimal module (separate `go.mod`) that vendors only the storage + observability code it needs, so gomobile sees a self-contained module with no `../` replaces. Largest blast radius; cleanest long-term.
2. **gomobile upstream fix / pin.** Try an older gomobile pin known to handle `replace` modules, or file/track the upstream bug.
3. **Manual cgo/JNI wrapper** bypassing gomobile `bind` entirely.

The exact reproduction commands are in this doc's "Usage" section; the diagnostic was `gomobile bind -work -target=android/arm64 ...` followed by inspecting the 0-byte `src-android-arm64/go.mod` in the printed `WORK=` dir.